

# DevOps para Devs

O básico que o dev precisa saber

25 minutos · Node.js como fio condutor

# O que veremos hoje

01



## Containers com Docker

Empacotar o app

02



## CI/CD na prática

Automatizar o caminho

03



## Ambientes e segredos

Separar config de código

04



## Docker Compose

multi-contêineres

05



## Observabilidade

Saber o que acontece em prod

# O projeto fio condutor

Node.js

Uma **API de tarefas** minimalista.



```
1 app.get('/tasks', (req, res) => res.json(tasks));
2 app.post('/tasks', (req, res) => {
3   const task = { id: tasks.length + 1, title: req.body.title, done: false };
4   tasks.push(task);
5   res.status(201).json(task);
6 });
```

# Bloco 1 — Containers com Docker

Empacotar o app de uma vez por todas



03:00 – 08:00

# O problema que Docker resolve

## ✗ Sem Docker

- ⚠ Node 18 na sua máquina
- ⚠ Node 20 no servidor
- ⚠ Lib instalada globalmente
- ⚠ "Funciona aqui..."



## ✓ Com Docker

- ✓ Ambiente definido no Dockerfile
- ✓ Igual em qualquer máquina
- ✓ Dependências isoladas
- ✓ "Roda em todo lugar"

# Dockerfile – a receita do container

## Dockerfile



```
1 FROM node:20-alpine
2 WORKDIR /app
3 COPY app/package*.json ./
4 RUN npm install
5 COPY app/ .
6 EXPOSE 3001
7 CMD ["node", "index.js"]
8
```

## Rodar o container:

```
docker build -t tasks-api .
docker run -p 3000:3000 tasks-api
```

# A Receita do Container Inquebrável

## A Base.

Traz um sistema operacional ultraleve já com a versão exata do Node.

## As Dependências.

Copia os arquivos e isola a instalação das bibliotecas.

```
1 FROM node:20-alpine
2 WORKDIR /app
3 COPY app/package*.json ./
4 RUN npm install
5 COPY app/ .
6 EXPOSE 3001
7 CMD ["node", "index.js"]
8
```

## A Ponte.

A porta pela qual este container isolado se comunica com o mundo exterior.

## A Ignição.

O comando exato que inicia a API.

**Construir & Rodar:** `docker build -t tasks-api . -> docker run -p 3000:3000 tasks-api`

# Bloco 2 — CI/CD na prática



Automatizar o caminho do commit ao deploy

08:00 – 13:00



# Do commit ao deploy – o pipeline



## CI – Integração Contínua

Roda automaticamente a cada push.  
Detecta problemas antes do merge.  
Feedback em minutos, não dias.

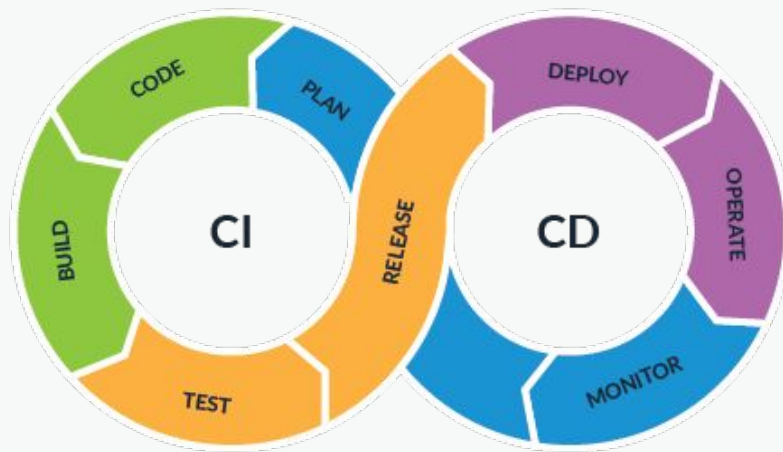
## CD – Entrega Contínua

Deploy liberado após testes passarem.  
Cada commit pode ir à produção.

# GitHub Actions — .github/workflows/ci.yml

Projeto

```
1  name: CI/CD
2
3  on:
4    push:
5      branches: [main]
6    pull_request:
7      branches: [main]
8
9  jobs:
10   test:
11     name: Testes
12     runs-on: ubuntu-latest
13     steps:
14       - uses: actions/checkout@v4
15       - run: make test
16
17   build-and-push:
18     name: Build e Push da Imagem
19     runs-on: ubuntu-latest
20     needs: test
21     environment: production
22     permissions:
23       contents: read
24       packages: write
```



# CI/CD na Prática (GitHub Actions)

**1. Gatilho**  
(Quando isso roda?)

```
1  name: CI/CD
2
3  on:
4    push:
5      branches: [main]
6    pull_request:
7      branches: [main]
8
9  jobs:
10   test:
11     name: Testes
12     runs-on: ubuntu-latest
13     steps:
14       - uses: actions/checkout@v4
15       - run: make test
16
17   build-and-push:
18     name: Build e Push da Imagem
19     runs-on: ubuntu-latest
20     needs: test
21     environment: production
22     permissions:
23       contents: read
24       packages: write
25
```

**2. O Escudo**  
(npm ci & npm test)

**3. A Esteira**  
(docker build & deploy)

# Bloco 3 — Ambientes e segredos

Separar configuração de código

13:00 – 17:00



# Nunca commite segredos — jamais

Projeto

`.env.example` — COMMITAR 

```
PORT=3000
DATABASE_URL=postgresql://user:password@localhost/t
asks
NODE_ENV=development
```

`.env` — NUNCA COMMITAR 

```
PORT=3000
DATABASE_URL=postgresql://admin:s3cr3t@db/tasks
NODE_ENV=production
```

## Development

Debug ativo, banco local, hot-reload

## Staging

Espelho de prod, testes de integração

## Production

Usuários reais — zero tolerância a erro

# Bloco 4 — Docker Compose



O básico que o dev precisa entender sobre deploy

17:00 – 21:00

# docker-compose.yml — app

Projeto

```
1  services:
2    app:
3      build: .
4      ports:
5        - "3001:3001"
6      depends_on:
7        - prometheus
8
9    prometheus:
10     image: prom/prometheus:latest
11     ports:
12       - "9090:9090"
13     volumes:
14       - ./prometheus.yml:/etc/prometheus/prometheus.yml
15
16    grafana:
17     image: grafana/grafana:latest
18     ports:
19       - "3000:3000"
20     environment:
21       - GF_SECURITY_ADMIN_PASSWORD=admin
22     volumes:
23       - ./grafana/provisioning:/etc/grafana/provisioning
24     depends_on:
25       - prometheus
26
```

## API de Tasks

Debug ativo, banco local, hot-reload

## Prometheus

Espelho de prod, testes de integração

## Grafana

Usuários reais — zero tolerância a erro

# A Anatomia do Docker Compose

```
1  services:
2    app:
3      build: .
4      ports:
5        - "3001:3001"
6      depends_on:
7        - prometheus
8
9    prometheus:
10     image: prom/prometheus:latest
11     ports:
12       - "9090:9090"
13     volumes:
14       - ./prometheus.yml:/etc/prometheus/prometheus.yml
15
16    grafana:
17     image: grafana/grafana:latest
18     ports:
19       - "3000:3000"
20     environment:
21       - GF_SECURITY_ADMIN_PASSWORD=admin
22     volumes:
23       - ./grafana/provisioning:/etc/grafana/provisioning
24     depends_on:
25       - prometheus
```

Nossa API (Node.js)  
exposta na porta 3001

Coletor de métricas  
(Dependência do App)

Dashboard visual  
(Dependência do Prometheus)



# Bloco 5 — Observabilidade



Saber o que está acontecendo em produção

21:00 – 25:00

# Log ruim vs log estruturado

Projeto

## ✗ Log ruim

```
app.post('/tasks', (req, res) => {
  try {
    tasks.push(req.body);
    res.status(201).json(req.body);
  } catch (err) {
    console.log('erro'); // inútil
    res.status(500).send('error');
  }
});
```

## ✓ Log estruturado (pino)

```
const logger = require('pino')();

app.post('/tasks', (req, res) => {
  try {
    tasks.push(req.body);
    logger.info({ task: req.body,
      userId: req.user?.id }, 'task criada');
    res.status(201).json(req.body);
  } catch (err) {
    logger.error({ err, body: req.body },
      'falha ao criar task');
  }
});
```

### Logs

Histórico estruturado (JSON)  
**Níveis: debug, info, warn, error**

### Métricas

**Latência, taxa de erro, CPU**  
Base para alertas

### Tracing

Requisição de ponta a ponta  
Cruzando múltiplos serviços

# 5 coisas para levar pra casa



## Dockerize desde o primeiro projeto

Acaba com o 'funciona na minha máquina'



## Pipeline de CI desde o início

Detecta bugs antes de virar problema



## Nunca commite segredo

Nenhuma exceção, nem em repo privado



## Saiba ler YAML e fazer rollback

kubectl rollout undo salva às 2h da manhã



## Escreva logs com contexto

Seu eu do futuro vai agradecer

# Obrigado!

DevOps para Devs — O básico que o dev precisa saber

---

Dúvidas?

Leituras recomendadas:

[12factor.net](https://12factor.net) · [docs.docker.com](https://docs.docker.com) · [kubernetes.io/docs](https://kubernetes.io/docs) · [getpino.io](https://getpino.io)